Plan: Cooperation-loop additions to `ask_local_oracle`

Context

The local LLM and hosted Claude don't currently work *together* -- Claude calls `ask_local_oracle`, the local LLM dumps prose + claims, Claude integrates and moves on. There is no loop, no exchange, no contribution Claude didn't already ask for. The local LLM has substrate vision (audit log, vault index, raw cached files) that Claude structurally cannot have, and surfaces none of it unless Claude pre-fetched it.

This plan turns the one-shot dump into a multi-pass cooperation loop by adding four fields to `OracleResponse`:

* **`related_substrate`** -- deterministic vault scan; the local layer surfaces themes / moments / failure-modes referencing the subjects in scope, that Claude didn't fetch. *Volunteer-substrate pattern.*
* **`next_best_calls`** -- LLM-generated; specific Tier-1 calls that would raise oracle confidence on the current question. Claude makes them and re-calls.
* **`unresolved_substrate`** -- LLM-generated; data the oracle would need fetched to compose confidently. Distinct from `next_best_calls` (specific tool suggestions) by being a higher-level "I'm missing X" signal.
* **`unresolved_intent`** -- LLM-generated; what the analyst should clarify before the oracle can compose. Distinguishes "fetch this" from "ask the analyst."

Plus a tool-description rewrite so Claude knows to act on these fields (iterate, fetch, clarify) rather than treat them as inert labels.

The substrate-vision asymmetry (local layer scans the vault, deterministically; LLM contributes reasoning about gaps) is the load-bearing case for why both layers exist. This is why this change is on the menu.

Out of scope (deliberate):
* A separate `challenge_my_claim` / `check_against_substrate` tool -- different trust-axis (substrate-consistency check on Claude's draft synthesis), file separately.
* Verifier hook on vault-write -- different placement, different ADR territory.
* ADR amendment to ADR 0022 -- write after the code lands, when the shape is concrete.
* Real Ollama smoke -- the v6.6.0 banner already defers this.

Approach

1. Extend `OracleResponse` (oracle.py)

Add four `list[...]` fields to `OracleResponse` and `to_dict()`. Top-level on the response (response content, not provenance -- per Phase 1 finding 5).

```
@dataclass
class OracleResponse:
    numerical_claims: list[NumericalClaim]
    narrative: str
    ambiguity_axes: list[str]
    confidence: float
    meta: OracleMeta
    # NEW (default empty for backward compatibility -- old call sites unaffected)
    related_substrate: list[dict] = field(default_factory=list)
    next_best_calls: list[str] = field(default_factory=list)
    unresolved_substrate: list[str] = field(default_factory=list)
```

```
    unresolved_intent: list[str] = field(default_factory=list)
```

`to_dict()` adds the four keys at top level. Each `related_substrate` entry shape:

```
{"kind": "theme" | "moment" | "failure_mode",
 "slug": str, "title": str | None, "subject_id": str | None,
 "status": str | None, "last_updated": str | None}
```

2. Wire `vault_storage` into `LocalLLMLayer` (layer.py)

Per Phase 1 recommendation (Option A -- explicit injection, mirrors `VaultLayer(vault_writer=...)` pattern):

```
class LocalLLMLayer:
    def __init__(self, backend=None, vault_storage=None):
        self._backend = backend or NullBackend()
        self._vault_storage = vault_storage  # may be None -- defensive
        self._router = None
```

In `execute()`, after `backend.compose(request)` returns, call a new helper `_scan_related_substrate(request)` to populate `response.related_substrate` regardless of which backend was used. This keeps the **substrate scan deterministic** (only LLM-driven fields come from the backend) and respects ADR 0008's determinism boundary.

```
async def execute(self, tool_name, params):
    # ... existing setup ...
    response = await self._backend.compose(request)
    response.related_substrate = self._scan_related_substrate(request)
    return response.to_dict()
```

`_scan_related_substrate()` shape:

* If `self._vault_storage is None`, return `[]`.
* Collect subject_ids from `request.subject_id` and any per-subject keys in `request.resolved_context` (the `{subject_id: {metric: value}}` shape `_flatten_claims` already detects in null.py:117-129).
* For each subject_id, call `VaultStorage.list_themes(subject_id=sid, limit=10)`, `list_notes(note_type='moment', subject_id=sid, limit=5)`, `list_notes(note_type='failure_mode', subject_id=sid, limit=5)`.
* Map each row -> the `related_substrate` entry shape above. Use `frontmatter.get('title')` for the title field (frontmatter is already on the dict per storage.py:200-252).
* Cap the total at 20 entries (token budget). Sort by `last_updated` desc.
* Catch and log any exception from VaultStorage, return `[]` -- never let a vault-scan failure break the oracle call.

3. Update backends for the LLM-driven fields

`NullBackend.compose()` (null.py): No prompt-engineering needed. Return empty lists for `next_best_calls`, `unresolved_substrate`, `unresolved_intent`. (NullBackend has no LLM; it cannot reason about gaps.) These default via the `field(default_factory=list)` on the dataclass -- *no code change needed* in NullBackend if defaults are present.

`OllamaBackend` (ollama.py):

* Update `_build_prompt()` (ollama.py:103-123) to extend the JSON schema with the three LLM-driven

fields. Update the system instruction to explain when to populate each.

* Update `_build_response()` (ollama.py:160-207) to parse the three new keys with defensive coercion to `list[str]` (matching the existing `ambiguity_axes` pattern at ollama.py:174-175).

* Update `_fallback_response()` (ollama.py:209-236) to set the three fields to `[]` and the unresolved ones to a single explanatory string each (e.g., `unresolved_intent = ["local guardian unavailable; cannot reason about gaps"]`).

4. Pass `vault_writer._storage` at registration (`__main__.py`)

Modify the registration block at `src/biosensor_mcp/__main__.py:210`:

```
router.register_local_llm_layer(LocalLLMLayer(
    backend=local_llm_backend,
    vault_storage=vault_writer._storage if _vault_path else None,
))
```

`_vault_path` truthiness check matches the existing pattern at `__main__.py:162` where the vault layer is conditionally registered.

5. Rewrite the tool description (layer.py:78-116)

Keep under 600 chars (current ~450; budget is comfortable per Phase 1 finding 6). Replacement description:

> *Ask the local-LLM guardian a natural-language question about already-computed analytical output. The framework pre-resolves the deterministic processing call(s) the question requires; the local LLM only composes a structured response. Numerical claims are citable (from processing.py); narrative is non-citable (LLM prose). Best paired with prior `csv_cohort_summary` / `csv_force_decline` calls -- pass their results as `resolved_context`. The response also returns `related_substrate` (vault notes the local layer found relevant -- incorporate, or call `vault_read_note` for full bodies), `next_best_calls` (Tier-1 calls that would raise confidence -- make them and re-invoke `ask_local_oracle` with the expanded context), and `unresolved_intent` (clarify with the analyst before proceeding). ~1500 tokens.*

6. Tests (`tests/framework/local_llm/test_local_llm.py`)

Add ~10 tests, matching the existing test-class structure (Phase 1 finding 4):

* `TestOracleResponseContract`:
- `test_to_dict_contains_collaboration_fields` -- all four fields present, default to `[]`.
- `test_to_dict_collaboration_fields_top_level` -- not nested under `_meta`.

* `TestNullBackend`:
- `test_null_backend_collaboration_fields_empty` -- NullBackend returns `[]` for the three LLM-driven fields.

* `TestOllamaBackend`:
- `test_compose_parses_next_best_calls` -- mocked Ollama returns JSON with `next_best_calls`; backend surfaces them.
- `test_compose_coerces_non_list_collaboration_fields` -- Ollama returns a string instead of list -> defensive coercion to `[]`.
- `test_fallback_response_collaboration_fields_explain_failure` -- on Ollama error, `unresolved_intent` carries the failure explanation.

  * `TestLocalLLMLayer` (new section `TestLocalLLMLayerSubstrateScan`):
 - `test_scan_related_substrate_returns_empty_when_no_vault` -- `vault_storage=None` -> `related_substrate == []`.
 - `test_scan_related_substrate_finds_themes_for_subject` -- fixture vault_storage returns one theme for P003 -> response includes it.
 - `test_scan_related_substrate_walks_per_subject_resolved_context` -- `resolved_context = {csv_cohort_summary: {P003: {...}, P004: {...}}}` -> scans for both subjects.
 - `test_scan_related_substrate_caps_total_entries` -- vault_storage returns 100 themes -> response has at most 20 entries.
 - `test_scan_related_substrate_swallows_storage_exception` -- vault_storage raises -> response.related_substrate is `[]`, no exception leaks.

  * `TestRouterDispatch`:
 - `test_dispatch_local_llm_preserves_collaboration_fields` -- end-to-end through router; all four fields present at result top level (not in `_meta.oracle`).

7. Update CLAUDE.md banner

Add a note under the v6.6.0 banner about the cooperation-loop extension, OR add a new patch banner (v6.6.1) once shipped. Defer to release time.

Files to modify

File	Change
src/biosensor_mcp/framework/local_llm/oracle.py	Add 4 fields to OracleResponse + to_dict()
src/biosensor_mcp/framework/local_llm/layer.py	vault_storage param; _scan_related_substrate() helper; execute() populates field; tool description
src/biosensor_mcp/framework/local_llm/backends/null.py	No code change (defaults handle it); confirm via test
src/biosensor_mcp/framework/local_llm/backends/ollama.py	Prompt extension; response parsing; fallback
src/biosensor_mcp/__main__.py	Pass vault_writer._storage at registration (line ~210)
tests/framework/local_llm/test_local_llm.py	~10 new tests across existing classes + new substrate-scan class

Existing utilities to reuse

  * `VaultStorage.list_themes(subject_id=..., limit=...)` -- `framework/vault/storage.py:410`
  * `VaultStorage.list_notes(note_type=..., subject_id=..., limit=...)` -- `framework/vault/storage.py:200`
  * `_flatten_claims` per-subject detection logic -- `framework/local_llm/backends/null.py:117-129` (the same shape detection feeds `_scan_related_substrate`)
  * Defensive list-coercion pattern -- `framework/local_llm/backends/ollama.py:174-175` (clone for the three new fields)
  * ADR 0009 IS-NULL-or-match filter semantics -- already implemented in `VaultStorage.list_themes`; we inherit it

Verification

End-to-end:

1. `pytest tests/framework/local_llm/ -v` -- new tests pass, existing 28+ tests still green.
2. `pytest -v` -- full suite still 632/632 (no cross-package regression).
3. `ruff check .` -- clean.
4. `biosensor-mcp demo` -- registration path runs without `vault_writer._storage` in scope (demo doesn't set up a vault); confirms the `if _vault_path else None` branch.
5. **Manual MCP wire smoke** (per project convention -- the mcp-protocol-auditor catches serialization regressions): drive `python -m biosensor_mcp serve` as a subprocess, send a `tools/call` for `ask_local_oracle` with a synthetic `resolved_context`, assert the response JSON contains `related_substrate`, `next_best_calls`, `unresolved_substrate`, `unresolved_intent` at top level (not under `_meta`), and that the values are lists. The `mcp-protocol-auditor` agent runs this as part of its standard sweep.
6. **Vault-substrate scan smoke**: with a configured vault containing one theme tagged `subject_id: P003`, call `ask_local_oracle` with `subject_id="P003"` and assert `related_substrate` is non-empty and contains the theme's slug.

Backwards compatibility

* New fields are additive on `OracleResponse` with `default_factory=list`. Existing callers reading `numerical_claims` / `narrative` / `_meta` are unaffected.

* Old Ollama JSON responses without the new keys -> defensive parsing returns `[]` for each (matches existing `ambiguity_axes` pattern).

* Mid-upgrade callers (call site upgraded but model still on old prompt) -> empty new fields, oracle still functional.

* No public API removal. SemVer-friendly minor (or patch) bump.

Deferred follow-ups (not in this plan)

* Tool-description nudge effectiveness -- the new description tells Claude how to handle the fields, but whether Claude actually iterates is a prompt-engineering question. Worth measuring after a few real sessions.

* ADR amendment to ADR 0022 (or new ADR) codifying the cooperation loop. Write after code lands; the shape is concrete then.

* Separate `verify_against_substrate` / `challenge_my_claim` tool for the substrate-consistency check on Claude's draft synthesis -- distinct from oracle composition.

* Releasing this as part of v6.6.1 patch or v6.7.0 minor -- release-shipper decides at ship time.